

Java Notes

Digital transcription of handwritten notes

Topics: static members, wrapper classes, access modifiers, initialization blocks, final keyword, this & super keywords, inheritance, constructors, constructor chaining, arrays

Note: In one file, two classes cannot be public.

Concepts Which Are Left

In Java

- (1) static member variable
- (2) static member functions
- (3) We cannot make a static variable inside a method
- (4) But we can make a static inner class

Code

```
public class Example
{
    int x;           // Instance variable
    static int y;     // Static member variable

    public void fun1() { }           // instance member method (can only be
                                    // called using object of that class)
    public static void fun2() { }    // static member method

    public void fun2()
    {
        static int z;    // X we cannot make static variable inside method
    }

    static class Test    // OK - static inner class
    {
    }
}
```

Static Variables

- (1) Static variables are declared in the class using the **static** keyword.
- (2) Static variables are by default initialized to their default value.
- (3) A static member has a single copy for the whole class & does not depend on the objects.

Let us understand points 2 & 3 with an example:

```
public class Example
{
    int x;
    static int y;        // static member variable

    public static void main(String[] args)
    {
        Example ex1 = new Example();    // every time, variable x will
        Example ex2 = new Example();    // be created

        Example.y = 5;    // now its default value changed
                           // static variable y is a single copy of class
    }
}
```

Static Method

- (1) Static methods defined inside the class are qualified with the keyword **static**.
- (2) Static methods can only access static members of the same class.
- (3) Static methods can be invoked using class name & dot operator.

Real Life Examples of Static Concepts

Account Number, Balance = instance variables

Rate of Interest = static variable

Code

```
class Example
{
    int x;
    static int y;
    private static int z;

    public void fun1()
    {
        x = 4;        // OK
    }

    public static void fun2()
    {
        // x = 4;    X   inside static method we cannot
        //                access instance variable
        y = 4;        // OK - we can access static variable
        z = 4;        // OK
    }

    public static void main(String args[])
    {
        y = 4;        // OK
        z = 4;        // OK
        // fun1();    X
        fun2();        // OK
    }
}

public class Hello
{
    public static void main(String args[])
    {
```

```
// y = 4;           X
Example.y = 4;      // OK

// z = 4;           X
// Example.z = 4;    X   (z is private) - but we can
//                  //   access z variable using method

// fun2();           X
Example.fun2();      // OK - now here z value will become 4
}
}
```

Static Class

- (1) We can have a class inside a class, which is known as an inner class.
- (2) An inner class can be qualified with the keyword **static**.

Code

```
class Example
{
    static class Test
    {
        public static String country = "India";
    }
}

public class Hello
{
    public static void main(String args[])
    {
        System.out.println(Example.Test.country);
    }
}
```

Wrapper Classes in Java

JAVA is 99% OOP.

- JAVA is an Object Oriented Programming language, & as said, everything in Java is an object.

Wrapper Classes

- As a solution to the problem of not being 100% OOP, Java allows you to include the primitives in the family of objects by using what are called **wrapper classes**.
- There is a wrapper class for every primitive data type in Java.
- This class encapsulates a single value for the primitive data type.
- For instance, the wrapper class for **int** is **Integer**, for **float** is **Float**, & so on.

Primitive	Wrapper Class
boolean	Boolean
byte	Byte
char	Character
short	Short
int	Integer
long	Long
float	Float
double	Double

Useful Methods of Wrapper Classes

(1) **valueOf()**

- Static method
- Returns object reference of the relative wrapper class

(2) **parseXxx()** (Xxx = primitive data type)

- Static method
- Xxx can be replaced by any primitive type
- It returns an Xxx type value

(3) **xxxValue()**

- Instance method of wrapper class
- Returns corresponding primitive type

Code Example

```
public class Example
{
    public static void main(String[] args)
    {
        Integer i1 = Integer.valueOf("101011", 2);
        Double d1 = Double.valueOf("3.14");

        int a = Integer.parseInt("123");
        double b = Double.parseDouble("13.45");

        int c = i1.intValue();
    }
}
```

Access Modifiers

⇒ Java supports four categories of accessibility rules:

- private
- protected
- public
- default

⇒ Modifiers can be used for class, member variables & member methods.

With Class

(1) Classes are divided into outer class & inner class.

(2) For an outer class, there can be only two possibilities: either the class is a **public** class, or just a class, which means it is of **default** type.

(3) For an inner class, any among the four access modifiers can be used.

Code

```
public class Example      // outer class
{
    class Dummy            // inner class
    {
    }
}
```


Remember

- (1) There can be only one public class in a single Java file.
- (2) The name of the Java file must be the same as the name of the public class.
- (3) Only a public class can be accessed directly from outside the package.

Initialization Block in Java

There are two types of initialization blocks:

→ Instance Initialization Block

→ Static Initialization Block

Instance Initialization Block

```
public class Test
{
    private int x;

    {    // Instance Initialization Block
        System.out.println("Initialization Block: x = " + x);
        x = 5;
    }

    public Test()    // <- block's code comes here (first)
    {
        System.out.println("Constructor: x = " + x);
    }

    public static void main(String args[])
    {
        Test t1 = new Test();
        Test t2 = new Test();
    }
}
```

Note: If there are more than one instance initialization block, then they all merge up & are added at the first of the constructor. If there is no constructor, then only instance initialization blocks run.

Important Points (Instance Initialization Block)

- (1) An instance initializer, or initialization block, declared in a class is executed when an instance of the class is created.
- (2) The **return** keyword cannot be used in an initialization block.
- (3) Instance initializers are permitted to refer to the current object via the keyword **this** & to use the keyword **super**.

Static Initialization Block

```
public class Test
{
    private static int k;

    static
    {
        System.out.println("Static Initialization Block: k = " + k);
        k = 10;          // only static things can be used
    }

    public static void main(String args[])
    {
        new Test();
    }
}
```

Note: Static initialization is automatically called when an object is created, or when calling any static or instance method of that class. For example: `Test.display()`;

Important Points (Static Initialization Block)

- (1) A static initializer declared in a class is executed when the class is initialized (when we are initializing or calling the method, it loads up).
- (2) Static initializers are used to initialize the static variables of the class.
- (3) The **return** keyword cannot be used in a static initialization block.
- (4) **this** or **super** cannot be used in a static block.

Method Overriding

```
class A
{
    public void f1(int x)
    {
        System.out.println("Class A");
    }
}

class B extends A
{
    public void f1(int x)
    {
        System.out.println("Class B");
    }
}

public class Example1
{
```

```
        public static void main(String args[])
        {
            B obj = new B();
            obj.f1(5);
        }
    }
```

Final Keyword

- (1) final instance variable
- (2) final static variable
- (3) final local variable
- (4) final class
- (5) final methods

Final Instance Variable

Three ways of initializing a final variable:

- during declaration
- inside initialization block
- inside constructor

```
public class Example
{
    private final int x;    // final instance member variable

    Example()
    {
        x = 5;
    }

    public static void main(String args[])
    {
        Example e1 = new Example();
    }
}
```

Note

- (1) A Java variable can be declared using the keyword **final**. Then the final variable can be assigned only once.
- (2) A variable that is declared as final and not initialized is called a **blank final variable**. A blank final variable forces either the constructor to initialize it, or the initialization block to do this job.

Final Static Variable

The **final** word can be written before **static** & after **static** also.

```
public class Example
{
    private final int x;
    private final static int y;

    Example()
    {
        x = 5;
    }

    static { y = 4; }

    public static void main(String args[])
    {
        Example e1 = new Example();
    }
}
```

Note

- (1) A static member variable, when qualified with the **final** keyword, becomes blank until initialised.
- (2) A final static variable can be initialized during declaration or within the static block.

Note: Local variables are blank variables, & final static local variables also.

Final Local Variable

```
public class Example
{
    public void fun()
    {
        final int k;    // final local variable
    }
}
```

Note

(1) Local variables that are final must be initialised before their use — but you should remember this rule is applicable to non-final variables too.

(2) Once they are initialized, they cannot be altered.

Final Class

A Java class declared as final cannot be extended — restricting inheritance!

```
final class Dummy
{
}
```

Final Method

Methods declared as final cannot be overridden.

[illegible]

this Keyword

```
class Box
{
    private int l, b, h;

    // instance member method
    public void setDimension(int l, int b, int h)
    {
        this.l = l;
        this.b = b;      // without 'this', ambiguity will occur
        this.h = h;
    }
}

public class Example
{
    // static member method
    public static void main(String args[])
    {
        Box b1 = new Box();
        b1.setDimension(12, 10, 5);
    }
}
```

Important Note

- (1) The **this** object reference is a local variable in instance member methods, referring to the caller object.
- (2) The **this** keyword is used as a reference to the current object, which is an instance of the current class.
- (3) The **this** reference to the current object is useful in situations where a local variable hides, or shadows, a field with the same name.

```
class Box
{
    private int l, b, h;

    public void setDimension(int l, int b, int h)
    {
        this.l = l;
        this.b = b;
        this.h = h;
    }

    public void sendBox()
    {
        GiftTaker gt = new GiftTaker();
        gt.acceptGift(this);
    }
}

public class Example
{
    public static void main(String args[])
    {
        Box b1 = new Box();
        b1.setDimension(12, 10, 5);
        b1.sendBox();
    }
}
```

Note

- (1) If a method needs to pass the current object to another method, it can do so using the **this** reference.
- (2) Note that the **this** reference cannot be used in a static context, as static code is not executed in the context of any object.

Q: Can **this** be used in a constructor?

Ans: Yes, because a constructor is an instance method.

Q: Can **this** be used inside main?

Ans: No, because main is a static method.

super Keyword

```
class A
{
    public void f1()
    { }
}

class B extends A
{
    public void f1()
    {
        super.f1();
    }
}

class Example
{
    public static void main(String args[])
    {
        B obj = new B();
        obj.f1();
    }
}
```

Diagram (from notes): *obj* → object of *B*; inside instance members, **this** refers to the current (*B*) object and **super** refers to the same object viewed as the superclass (*A*) part.

Note

(1) In inheritance, when a subclass object calls an instance member method of the subclass, the method contains implicit reference variables **this** and **super**, both referring to the current object (object of subclass).

(2) The only difference between **this** and **super** is:

- (a) the **this** reference is of subclass type;
- (b) the **super** reference is of superclass type.

Use of super Keyword

(1) If your method overrides one of its superclass's methods, you can invoke the superclass version of the method through the use of the keyword **super**.

(2) It avoids name conflict (ambiguity) between members of superclass & subclass.

```
class A { int z; }

class B extends A
{
    int z;

    public void f2()
    {
        int z;
        super.z = 2;    // superclass field
        this.z  = 3;    // subclass field
        z       = 4;    // local variable
    }
}
```

```
class Example
{
    public static void main(String args[])
    {
        B obj = new B();
        obj.f2();
    }
}
```

Static Members in Inheritance

```
class Parent
{
    public static void f1()
    {
        System.out.println("Hello");
    }
}

class Child extends Parent
{
}

public class Example
{
    public static void main(String args[])
    {
        Child.f1();
    }
}
```

Points

- (1) A class C inherits from its direct superclass all concrete methods m (both static & instance) of the superclass.
- (2) No method declared in C has the same signature.

```
class Parent
{
    public static void f1()
    {
        System.out.println("Hi, I am from Parent");
    }
}

class Child extends Parent
{
    public static void f1()
    {
        System.out.println("Hi, I am from Child");
    }
}

public class Example
{
    public static void main(String args[])
    {
        Child.f1();
    }
}
```

Method Hiding

If a subclass has a method *m* with the same signature as of the method present in the super class, then method *m* **hides** the method of the superclass.

Remember

(1) It is a compile-time error if a static method hides an instance method.

(2) It is a compile-time error if an instance method overrides a static method.

Static Member Variables

```
class Parent
{
    static int y = 4;
}

class Child extends Parent
{
    static
    {
        y = 5;
    }
}

public class Example
{
    public static void main(String args[])
    {
        System.out.println("y = " + Child.y);    // y = 4
    }
}
```

Note: Static member variables do not inherit, but static member methods do.

Constructors in Inheritance

(1) Constructors are not inherited by a class.

Q: What happens when an object of a subclass is created?

Q: What is the role of a constructor?

```
class A
{
    int a;
    public A()
    {
        System.out.println("A");
    }
}

class B extends A
{
    int b;
    public B()
    {
        super();    // should be written in first line of constructor;
                   // if you do not write it, the compiler will
        System.out.println("B");
    }
}

class Example
{
    public static void main(String args[])
    {
        B obj = new B();
    }
}

/* Output:
A
B
*/
```

Point

A subclass's constructor invokes the constructor of the super class.

Code

```
class A
{
    int a;
    public A(int x)
    {
        a = x;
        System.out.println("A");
    }
}

class B extends A
{
    int b;
    public B()
    {
        super(4);
        System.out.println("B");
    }
}

class Example
{
    public static void main(String args[])
    {
        B obj = new B();
    }
}
```

Points

- (1) An explicit call to the super class constructor from the sub class's constructor can be made using **super()**.
- (2) You can write a subclass constructor that invokes the constructor of the superclass, either implicitly or by using the keyword **super**.

Scenarios

- (1) Implicit constructors in superclass & subclass.
- (2) Implicit constructor in subclass and explicit constructor in superclass.
- (3) Implicit constructor in superclass & explicit constructor in subclass.
- (4) Explicit constructor in superclass & subclass.

Constructor Chaining in Java

```
class A
{
    public A()
    {
        System.out.println("A1");
    }
}

class B extends A
{
    public B()
    {
        this(4);    // it will call constructor of same class
                   // with 1 argument
        System.out.println("B1");
    }

    public B(int x)
    {
        System.out.println("B2");
    }
}

class Example
{
    public static void main(String args[])
    {
        B obj = new B();
    }
}

/* Output:
A1
B2
B1
*/
```

Points

- (1) A constructor can call another constructor of the same class or the super class.
- (2) A constructor call from a constructor must be the first step (the call should appear in the first line).
- (3) A series of invocations of constructors is known as **constructor chaining**.

Arrays in Java

```
int arr[5];    // error
```

```
int arr[];    // reference variable of array
```

So, proper syntax of making an array — it is similar to making an object:

```
int arr[] = new int[5];
```

```
// or we can change position of brackets  
int[] arr = new int[5];
```

Memory Allocation of Array in Java

arr (reference variable) → points to a block of 5 cells (indexes 0–4) with a **length** property = 5.

```
arr[0] = value;  
arr.length == 5
```

Initialization at the Time of Declaration

```
int arr[] = new int[] {2, 4, 6, 8, 10};  
// or  
int arr[] = {2, 4, 6, 8, 10};
```

Some Common Mistakes & Errors

```
(1) int arr[] = new int[2] {2, 4, 6, 8, 10};    // error
```

```
(2) int arr[] = new int[5] {2, 4, 6, 8, 10};    // error  
    // you cannot mention size & values together
```

Note: An array is not blank — if we don't initialize an array, then by default the value 0 will be put inside it.

```
int arr[] = new int[3];  
System.out.println("arr[0] " + arr[0]);  
// Output: arr[0] 0
```

Find Error in Program?

```
public class ArrayExample  
{  
    public static void main(String args[])  
    {  
        int arr[];           // we have not allocated the array,  
        arr[0] = 25;         // so it will act as a  
        arr[1] = 50;         // local blank variable -> error  
    }  
}
```

Initialize After Declaration

```
public class ArrayExample  
{  
    public static void main(String args[])  
    {  
        int arr[];  
        arr = new int[3];  
        arr[0] = 10;  arr[1] = 20;  arr[2] = 30;  
    }  
}
```

Two Dimensional Array

```
int arr[][];  
int [][]arr;
```

```
// So,  
int [][]a = new int[4][5];
```

Memory picture (from notes): a is a reference to an array of 4 references (indexes 0–3), each pointing to its own row array of 5 ints.

Note

```
(1) int a[][] = new int[][];    // error  
(2) int a[][] = new int[][5];  // error  
(3) int a[][] = new int[4][];   // OK
```

Initializing 2D Array at Declaration

```
int a[][] = new int[][] { {3,4}, {5,6}, {7,8} };
```


Find Error

```
public class Example
{
    public static void main(String args[])
    {
        int[][] a = new int[4][];
        System.out.println("a[0][0] " + a[0][0]);
    }
}
// Error: NullPointerException
```

Use of length Property

```
public class Example1
{
    public static void main(String args[])
    {
        int [][]a = new int[4][];
        System.out.println("Length = " + a.length);
    }
}
// Output: 4
```

Find Output

```
public class Example1
{
    public static void main(String args[])
    {
        int [][]a = new int[4][];
        System.out.println("Length = " + a[0].length);
    }
}
```

Example: User Input

```
import java.util.*;

class ArrayExample1
{
    public static void main(String args[])
    {
        int arr[] = new int[5];
        Scanner scanner = new Scanner(System.in);

        for (int i = 0; i < 5; i++)
        {
            arr[i] = scanner.nextInt();
        }

        for (int i = 0; i < 5; i++)
        {
            System.out.println("arr" + (i+1) + " " + arr[i]);
        }
    }
}
```

2D Array with Different Sizes

```
public class Example
{
    public static void main(String args[])
    {
        int a[][] = new int[4][];
        a[0] = new int[5];
        a[1] = new int[3];
        System.out.println("Length = " + a[1].length);
    }
}
```